Submission guidelines:
- Present complete, self-contained solutions.
- For SAT encodings: clearly define variables, constraints (in English and as clauses/formulas), and report the final clause count.
- For hands-on solver problems: include the DIMACS CNF file and the solver output.
- For DPLL/CDCL traces: show every step — decision, propagation, conflict, backtrack/backjump.
- For programming tasks: submit a code repository (GitHub/GitLab) with a README.

Problem 1: Warm-Up — Encoding Basics

This problem revisits the fundamental cardinality encoding patterns from lecture: ALO, AMO, and EO. Use pairwise AMO throughout (no auxiliary variables needed for small n).

(a) **Pigeonhole encoding.** Write the complete DIMACS CNF encoding of PHP_4^3 — 4 pigeons, 3 holes (pigeon i in hole j is variable $p_{i,j}$, numbered row-major: $p_{1,1} = 1$, $p_{1,2} = 2$, ...). List every clause explicitly. Then, check with a SAT solver: paste the CNF into CaDiCaL (or any solver) and report the result. Explain why this instance is necessarily UNSAT even before running the solver.

(b) **Encoding size analysis.** For the general PHP_{n+1}^n encoding:
- How many variables does it use?
- How many ALO clauses?
- How many AMO clauses?
- Give closed-form expressions and verify for $n = 3$.

(c) **Sudoku (mini).** A 4×4 Sudoku grid has 16 cells arranged in a 4×4 matrix, with four 2×2 boxes. Each cell gets a value $1, 2, 3$, or 4. The *standard Sudoku constraints* are: each value appears exactly once in each row, column, and 2×2 box.
- Define your variables (name them clearly).
- Write the EO constraints for one row, one column, and one box in clause form.
- What is the total clause count for the full 4×4 Sudoku encoding?

(You do not need to solve it with a solver.)

Problem 2: DPLL and CDCL Traces

Work through the following formulas by hand, simulating DPLL and CDCL.

(a) **DPLL trace.** Consider F_1 over $\{A, B, C, D\}$:

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge (\neg C \vee D) \wedge (\neg C \vee \neg D)$$

Run DPLL starting with $A = 1$. Show the full trace: at each step state the current partial assignment, the set of remaining clauses, and the action taken (decision, unit propagation, or backtrack). Does the algorithm find a satisfying assignment or prove UNSAT?

(b) **CDCL trace with conflict analysis.** Consider F_2 over $\{x_1, x_2, x_3, x_4, x_5\}$:

$$c_1 = (x_1 \vee x_2), \quad c_2 = (\neg x_1 \vee x_3), \quad c_3 = (\neg x_2 \vee x_4),$$
$$c_4 = (\neg x_3 \vee \neg x_4), \quad c_5 = (\neg x_3 \vee x_5), \quad c_6 = (\neg x_4 \vee \neg x_5)$$

Run CDCL with the following decision sequence: at decision level 1, decide $x_1 = 1$; at decision level 2, decide $x_2 = 1$.

- Record all unit propagations forced at each level.
- Draw the implication graph at the point of conflict.
- Identify the 1-UIP and compute the learned clause.
- State the backjump level and continue the trace to termination.
- Report the final satisfying assignment.

(c) **Comparing DPLL and CDCL.** Run *both* DPLL and CDCL on the pigeonhole formula PHP_3^2 (3 pigeons, 2 holes) from scratch.

- How many decisions does each algorithm make?
- How many backtracks/backjumps?
- Explain informally why CDCL outperforms DPLL on pigeonhole instances.

Problem 3: Standard SAT Encodings

Each part asks you to encode a combinatorial problem as SAT. For each encoding:

- State the variables with their intended meaning.
- State all constraints in English, then as propositional clauses (or clear clause schemas).
- Give the total variable and clause count as a function of the problem parameters.

(a) **Graph 3-coloring.** Encode the problem "is graph $G = (V, E)$ 3-colorable?" as SAT. Test your encoding on the *Petersen graph* (10 vertices, 15 edges).

(b) **Hamiltonian path.** A *Hamiltonian path* in an undirected graph is a path that visits every vertex exactly once. Encode the existence of a Hamiltonian path as SAT. *Hint:* introduce position variables $p_{v,i}$ meaning "vertex v is at position i in the path."

- Express "each position is occupied by exactly one vertex."
- Express "each vertex appears at exactly one position."
- Express "consecutive positions in the path are connected by an edge."

(c) **Latin squares.** A *Latin square* of order n is an $n \times n$ grid filled with n different symbols, each occurring exactly once in each row and column. Encode the existence of a Latin square of order n as SAT. (Hint: variables $L_{i,j,v}$ = "cell (i, j) has symbol v.")

- What is the variable count and clause count for general n?
- Check your encoding for $n = 3$: generate the CNF and run a solver to find a Latin square.

(d) **Exact cover.** Given a universe $U = \{1, 2, 3, 4, 5, 6, 7\}$ and a collection of subsets

$$S_1 = \{1, 4, 7\}, \quad S_2 = \{1, 4\}, \quad S_3 = \{4, 5, 7\},$$
$$S_4 = \{3, 5, 6\}, \quad S_5 = \{2, 3, 6, 7\}, \quad S_6 = \{2, 7\}.$$

An *exact cover* is a sub-collection $S' \subseteq \{S_1, ..., S_6\}$ such that every element of U appears in *exactly one* set in S'.

- Introduce Boolean variables $x_1, ..., x_6$ for set inclusion.
- For each element $u \in U$, write a clause enforcing "exactly one selected set contains u."
- Run your encoding with a SAT solver and report the exact cover found (or prove there is none).

Problem 4: Synthesis via SAT — Combinatorial Structures

The following problems ask you to *find* or *prove impossible* a combinatorial object by searching for it with a SAT solver. These are pure finite-domain synthesis problems.

(a) **Ramsey numbers — $R(3, 3) = 6$.** Color the edges of the complete graph K_n red or blue. A *monochromatic triangle* is a triangle all of whose edges share the same color.

- Encode "there exists a 2-coloring of K_5 with no monochromatic triangle" as SAT. Run the solver: is it SAT or UNSAT?
- Encode the same property for K_6. Run the solver.
- This establishes $R(3, 3) = 6$. Explain: what does it mean for the encoding of K_5 to be SAT and K_6 to be UNSAT? What coloring do you recover from the K_5 model?

(b) **Self-complementary graphs.** A graph G on n vertices is *self-complementary* if G is isomorphic to its complement $\overline{G}$.

- Encode the existence of a self-complementary graph on $n = 4$ vertices: find a labeling σ of vertices (a permutation) and an edge set E such that E is exactly the set $\{(\sigma(u), \sigma(v)) : (u, v) \notin E\}$. *(Hint: introduce edge variables $e_{i,j}$ for $i < j$ and permutation variables $\pi_{i,j}$ for "vertex i maps to vertex j.")*
- Run your encoding. Report the self-complementary graph found and verify it by hand.

(c) **Non-attacking rooks with forbidden squares.** Place n non-attacking rooks on an $n \times n$ board where some squares are forbidden. Specifically, for $n = 6$, the forbidden squares are the main diagonal: (i, i) for all i.

- Encode this as SAT.
- Find a valid placement and draw it.
- How does this relate to the existence of a derangement (a permutation without fixed points)?

(d) **Magic squares (small).** A *magic square* of order n is an $n \times n$ grid containing the integers 1 through n^2, each exactly once, such that all rows, columns, and both main diagonals sum to the same value (the *magic constant* $M = \frac{n(n^2+1)}{2}$).
- For $n = 3$ ($M = 15$): encode the existence of a magic square. Variables: $x_{i,j} \in \{1, ..., 9\}$ — but since SAT works with bits, represent each cell as a 4-bit number $b_{i,j,0}, ..., b_{i,j,3}$ and encode the binary constraints for "all distinct" and "row/column/diagonal sums equal 15."
- Run a solver and report a solution.
- *Alternative:* if binary arithmetic clauses are tedious, use a pseudo-Boolean solver (*e.g.*, PBLib + CaDiCaL) or encode the sum constraint with a network of adder circuits.
- Explain the trade-off: bitwise encoding vs. pseudo-Boolean encoding.

Problem 5: DFA Synthesis from Examples

A *deterministic finite automaton* (DFA) for an alphabet $\Sigma = \{a, b\}$ consists of:
- A finite set Q of states, a start state $q_0 \in Q$, a set $F \subseteq Q$ of final states, and a transition function $\delta : Q \times \Sigma \to Q$.

It *accepts* a word w if the run of δ on w starting from q_0 ends in a state in F.

Given a finite set of *positive examples* P^+ (words the DFA must accept) and *negative examples* P^- (words it must reject), *DFA synthesis* asks: does a DFA with k states exist that correctly classifies all examples?

Example set: $\Sigma = \{a, b\}$, $k = 2$ states, and $P^+ = \{\varepsilon, \text{aa}, \text{bb}, \text{abba}\}$, $P^- = \{\text{a}, \text{b}, \text{ab}, \text{ba}\}$.

The DFA should accept words of *even length* and reject words of *odd length*.

(a) **Variable design.** For a DFA with k states and alphabet $\Sigma = \{a, b\}$, introduce:
- $q_0 \in \{0, ..., k-1\}$ — start state (constant or variable?);
- f_s = "state s is a final state," for $s \in \{0, ..., k-1\}$;
- $\delta_{s,a,t}$ = "on input a in state s, go to state t," for $s, t \in \{0, ..., k-1\}$, and similarly for b.

State how many Boolean variables each group contributes.

(b) **Transition function constraints.** The transition function must be a *total function*: for each (s, σ), exactly one t satisfies $\delta_{s,\sigma,t}$. Write these EO constraints in clause form.

(c) **Run encoding.** For a word $w = w_1 w_2 \cdots w_m$, a *run* is a sequence of states $r_0, r_1, ..., r_m$ with $r_0 = q_0$ and $r_i = \delta(r_{i-1}, w_i)$. Introduce *run variables* $R_{w,i,s}$ = "run of w is in state s after position i." Write the clauses that enforce:
- At each position, exactly one state is active.
- The run starts in the start state.
- Each step respects the transition function ($R_{w,i-1,s} \wedge \delta_{s,w_i,t} \to R_{w,i,t}$).
- Words in P^+ end in a final state; words in P^- end in a non-final state.

(d) **Solving.** Instantiate the encoding for $k = 2$ and the example set above. Run a SAT solver. From the model, read off the DFA's transition table and draw its diagram. Verify by hand that it correctly classifies all examples.

(e) **Minimality.** First try $k = 1$. Is the formula SAT? Why or why not? What is the minimum k for which synthesis succeeds for this example set?

Problem 6: Boolean Formula Synthesis

A *monotone Boolean circuit* (or *formula tree*) over variables $x_1, \ldots, x_n$ consists of:

- *Leaf nodes:* each is a variable x_i or a constant 0/1.
- *Internal nodes:* each is an AND ($\wedge$) or OR ($\vee$) gate with exactly two children.
- One designated *root* node whose output is the formula's value.

The *size* of the circuit is the number of gates (internal nodes).

(a) **Majority function.** The *majority function* $\mathrm{maj}(x, y, z)$ returns 1 iff at least two of x, y, z are 1. Its truth table: $000 \mapsto 0$, $001 \mapsto 0$, $010 \mapsto 0$, $011 \mapsto 1$, $100 \mapsto 0$, $101 \mapsto 1$, $110 \mapsto 1$, $111 \mapsto 1$. Encode: "does a monotone Boolean formula of size ≤ 4 compute exactly the majority function?"

Hint: Introduction of variables for the gate type ($G_i = 0$ for AND, $G_i = 1$ for OR), left/right child pointers (one-hot variables for which node/leaf the child is), and output variables for each gate under each input combination.

- What is the minimum formula size to compute majority?
- Run a solver for size 4 and report the formula found.

(b) **Threshold function T_2^4.** $T_k^{n(x_1,\ldots,x_n)} = 1$ iff at least k of the x_i are 1. For T_2^4 (at least 2 of 4 variables are true):

- Write down its truth table (16 rows).
- Encode "does a monotone formula of size ≤ 6 compute T_2^4?"
- Run a solver and report a formula if one exists, or prove impossibility otherwise.

(c) **Formula vs. circuit.** A *circuit* (DAG) can share subexpressions; a *formula tree* (tree) cannot. The parity function $\mathrm{XOR}(x_1, x_2, x_3, x_4)$ requires 8 binary gates as a formula but only 4 as a circuit.

- *(Conceptual, no solver needed.)* Explain why tree-based formulas must repeat subexpressions where circuits can share.
- Does this affect our SAT encoding? If we encode tree-based synthesis, why can formulas for XOR be exponentially larger than circuits?

(d) **Complete Boolean basis.** Extend the synthesis question to include NOT gates (full Boolean basis: AND, OR, NOT). Encode the synthesis of $\mathrm{XOR}(x, y) = (x \vee y) \wedge (\neg x \vee \neg y)$ using a formula of size ≤ 4 (using AND, OR, NOT gates). Run a solver and verify the result.

Problem 7: Graph Synthesis and Verification

SAT solvers can *search* for graphs satisfying structural properties and *verify* impossibility.

(a) **Vertex cover.** A *vertex cover* of a graph $G = (V, E)$ is a subset $S \subseteq V$ such that for every edge $(u, v) \in E$, at least one of u, v is in S. The *minimum vertex cover* has the smallest possible size.

Fix the graph: $V = \{1, ..., 7\}$ with edges {1–2, 1–3, 2–4, 3–4, 4–5, 5–6, 5–7}.
- Encode the question "does G have a vertex cover of size $\leq k$?" as SAT.
- Binary search on k: run the solver for $k = 2, 3, 4$ until you find the minimum.
- Report the minimum vertex cover and the SAT/UNSAT outcomes.

(b) **Clique number.** A *clique* is a complete subgraph. Encode "does G contain a clique of size $\geq k$?" for the same graph as above.
- Run for $k = 3$ and $k = 4$.
- Report the omega number $\omega(G)$ (max clique size).

(c) **Graph property isomorphism.** Two graphs G and H on n vertices are *isomorphic* if there exists a bijection $\sigma : V(G) \to V(H)$ such that $(u, v) \in E(G) \Leftrightarrow (\sigma(u), \sigma(v)) \in E(H)$. Given:
- G: the cycle C_6 (vertices $1, ..., 6$ in a ring).
- H: two disjoint triangles $K_3 \cup K_3$ (vertices $\{1, 2, 3\}$ all connected and $\{4, 5, 6\}$ all connected, no edges between them).

Encode graph isomorphism as SAT:
- Variables: $\pi_{i,j}$ = "vertex i of G maps to vertex j of H."
- Constraints: (i) π is a bijection (ALO + AMO per row and column); (ii) edge preservation.
- Run the solver. Are G and H isomorphic? Justify.

(d) **Ramsey $R(3, 3, 3)$.** *(Challenge.)* Color the edges of K_n with 3 colors. A monochromatic triangle is a triangle all of whose edges share the same color. The Ramsey number $R(3, 3, 3)$ is the smallest n such that every 3-coloring of K_n contains a monochromatic triangle. It is known that $R(3, 3, 3) = 17$.
- Encode: "does a 3-coloring of K_{16} exist with no monochromatic triangle?"
- Run a solver (this may take seconds to minutes for K_{16}).
- Report the size of your CNF (variables, clauses) and the time taken.

(Bonus: visualize the coloring you find.)

Problem 8: Constraint Propagation and Resolution Theory

(a) **Unit propagation completeness.** Consider the formula $F = (A \vee B) \wedge (\neg A \vee B) \wedge (\neg B \vee C) \wedge (\neg C)$.
- Run unit propagation to completion (no decisions). Report the propagated assignments and the resulting formula at each step.
- Does unit propagation alone suffice to determine satisfiability? If not, which variable must be decided?
- After deciding that variable, run unit propagation again and determine the final status.

(b) **Resolution refutation.** Use binary resolution to derive the empty clause from:

$$\Gamma = \{(A \vee B), (\neg A \vee C), (\neg B \vee C), (\neg C)\}$$

Annotate each resolvent: (C_i) resolved with (C_j) on literal L gives (C_k). How many resolution steps are needed?

(c) **Size of resolution proofs.** The formula PHP_3^2 (3 pigeons, 2 holes) has 6 variables and 9 clauses.

- Write out all 9 clauses explicitly.
- Derive the empty clause by resolution, showing all steps.
- Exponential lower bounds for pigeonhole: resolution proofs of PHP_{n+1}^n require $2^{\Omega(n)}$ steps. Explain in your own words why this is the case and what it implies for DPLL solvers.

(d) **CDCL dominance.** The formula constructed in (c) takes DPLL exponential time. Explain (informally but precisely) how CDCL with 1-UIP learning avoids this blowup on PHP_3^2. *(Hint: what is the first learned clause after the first conflict, and how does it prune the search tree compared to chronological backtracking?)*

Problem 9: SAT Phase Transition and Random Instances

Random k-SAT instances with n variables and m clauses (each clause is a uniformly random set of k distinct literals, each variable negated independently with probability $\frac{1}{2}$) exhibit a *phase transition*: for $k = 3$, the formula is almost certainly satisfiable if $\frac{m}{n} < 4.267$ and almost certainly unsatisfiable if $\frac{m}{n} > 4.267$.

(a) **Empirical phase transition.** Write a script (Python or any language) that:

- Generates random 3-SAT instances with $n = 50$ variables and clause ratios $\frac{m}{n} \in \{3.0, 3.5, 4.0, 4.267, 4.5, 5.0, 6.0\}$.
- For each ratio, generates 100 independent instances and solves each with CaDiCaL (via subprocess or the `pycadical` binding).
- Plots the empirical satisfiability probability vs. ratio.

Compare your plot to the theoretical prediction; explain any discrepancies.

(b) **Solver time at the threshold.** Extend the experiment: record the solver runtime (in milliseconds) for each instance. Plot median runtime vs. ratio. Where does the peak occur? Why is solving hardest at the phase transition?

(c) **Scaling behavior.** Repeat the median-runtime experiment for $n \in \{50, 100, 150, 200\}$ at the critical ratio $\frac{m}{n} = 4.267$. Fit the growth curve. Is it consistent with polynomial or exponential scaling in practice (for these sizes)?

Problem 10: Programming Challenge — SAT Solver and Applications

Build SAT-solving tools from scratch. Choose your language: Python, Rust, OCaml, Haskell, C++, or any language with good data structures.

Submission: Code repository with a README describing your design, plus a brief report (2–3 pages) covering what you implemented, challenges encountered, and insights gained.

Part A: DPLL Solver

Task A.1: Core DPLL Implementation

Required:

- Represent CNF formulas as lists of clauses; clauses as lists of literals (signed integers).
- Implement unit propagation: repeatedly find unit clauses and force the assignment.
- Implement pure literal elimination.
- Implement the DPLL recursive procedure with backtracking.

Interface:

```python
def dpll(clauses: list[list[int]]) -> dict[int, bool] | None:
    """Return a satisfying assignment or None if UNSAT."""
```

Test cases:

- Solve all examples from Problem 2.
- Verify on `PHP_3^2` (should return UNSAT).
- Verify on your K_5 Ramsey encoding (should return SAT with a coloring).

Task A.2: DIMACS Parser

Required:

- Parse DIMACS CNF format into your internal representation.
- Handle comments (`c ...`) and the header `p cnf <nvars> <nclauses>`.

Test: Parse the PHP_4^3 instance you generated in Problem 1(a) and solve it.

Part B: CDCL Extensions

Task B.1: Implication Graph and Conflict Analysis

Required:

- Track reasons for each propagation: store the *reason clause* for each forced literal.
- On conflict, trace the implication graph backward to compute the 1-UIP.
- Generate the learned clause from the cut induced by the 1-UIP.
- Implement non-chronological backjumping to the correct level.

Note: Decision levels must be tracked per variable; a trail data structure (list of assigned literals in order) simplifies backjumping.

Task B.2: VSIDS Heuristic

Required:
- Maintain an activity score for each variable.
- Bump activity of all variables appearing in each learned clause.
- Decay all activity scores by factor 0.95 after each conflict.
- Use the highest-activity unassigned variable as the next decision.

Extension: Compare CDCL with random variable ordering vs. VSIDS on your Ramsey K_{16} encoding from Problem 7(d). Report the difference in conflicts and runtime.

Part C: Applications

Task C.1: DFA Synthesizer

Build a DFA synthesizer using your SAT solver (or an external solver) following the encoding from Problem 5.

Interface:

```python
def synthesize_dfa(
    pos: list[str],
    neg: list[str],
    alphabet: list[str],
    max_states: int,
) -> DFA | None:
    """Return the smallest DFA consistent with examples, or None."""
```

Test: Synthesize DFAs for:
- Even-length words over $\{a, b\}$.
- Words over $\{a, b\}$ ending in ab.
- Words over $\{0, 1\}$ where the number of 1s is divisible by 3.

For each, verify the synthesized DFA is correct on additional test inputs.

Task C.2: Constraint-Based Puzzle Solver

Implement a general puzzle solver using SAT. Support at least one of the following puzzles:

Option 1 — Nonogram (Picross): Given row and column clue sequences (counts of consecutive filled cells), find the grid. Encode each row/column clue as a set of SAT clauses (transition-based or positional encoding).

Option 2 — Slitherlink: Given a grid of number hints (0, 1, 2, 3, or blank) in a rectangular grid, draw a single closed loop along grid edges. Each numbered cell is adjacent to exactly that many loop segments. Encode: edge variables, loop-degree constraints, and single-loop connectivity (the hardest part — consider encoding connectivity as reachability in a spanning tree).

Option 3 — Cryptarithmetic: Solve puzzles of the form $\text{SEND} + \text{MORE} = \text{MONEY}$: assign distinct digits 0–9 to letters such that the equation holds. Encode addition carry chains as CNF (4-bit adders).

In all cases: document your encoding, solve at least 3 non-trivial instances, and report solution times.

Part D: Formal Verification Track (Optional)

Task D.1: Unit Propagation Correctness (Lean/Coq)

Formalize unit propagation in Lean 4 or Coq:

- Define CNF as a list of clauses.
- Implement `unit_propagate : CNF → Assignment → CNF × Assignment`.
- **Prove:** unit propagation is sound — if F is satisfiable with assignment ν and we propagate literal ℓ, then the simplified formula $F|_\ell$ is satisfiable with the restriction $\nu[x \mapsto \ell]$.
- **Prove:** unit propagation is complete for unit resolution — the simplified formula is unsatisfiable iff F is unsatisfiable given ℓ.

Task D.2: DRAT Proof Checker (Lean/Coq)

CDCL solvers can emit *DRAT proofs* — certificates of unsatisfiability as a sequence of clause additions and deletions. A DRAT proof is checked by verifying each addition step has the *RAT property* (Resolution Asymmetric Tautology).

- Implement a DRAT checker in your chosen language.
- *(Lean/Coq bonus):* Prove your checker is sound: if it accepts a DRAT proof of Φ, then Φ is indeed unsatisfiable.
- Test on the UNSAT output of CaDiCaL for PHP_4^3.